Dossier de préparation

# Pocket stars

Exploration interactive 3D des cartes Pokémon

Camélia Amin Handoyo

2026

# Sommaire


1. Introduction 3

2. Objectifs du projet 3

3. Choix de l'API 4

5. Choix techniques 4

6. Moodboard 5

7. Arborescence de navigation 6

8. Maquettes 7

9. Schéma d'intégration des données de l'API 8

10. Contrat de l'API 9

11. Planning 10

12. Risques et marges de sécurité 10

# Contexte

Ce dossier présente la phase de préparation du projet ***Pocket stars***, réalisé dans le cadre du module de de datavisualisation créative.

La mission confiée est de concevoir, en tant que développeuse front-end junior au sein d'une agence web fictive spécialisée en datavisualisation, une application web interactive combinant React, Three.js et une API publique, pour explorer visuellement un jeu de données en trois dimensions.

***Poket stars*** revisite l'univers de *Pokémon*, présentant un ciel de pluie d'étoiles qui représentent chacune une carte.

## Objectifs du projet

Le cahier des charges demande une navigation fluide permettant d'interagir avec les données (filtrer, survoler, zoomer, cliquer), au sein d'une interface responsive et visuellement attrayante.

Pour ***Pocket stars***, cet objectif se traduit concrètement par :

- Transformer chaque carte de l'API en étoile explorable dans un ciel en 3D, plutôt qu'en simple liste ou grille ;
- Permettre de retrouver une carte précise par son nom, sa date de révélation ou son numéro ;
- Révéler chaque carte par une courte animation au clic, comme on retournerait une vraie carte à jouer ;
- Accompagner l'exploration d'une ambiance sonore et d'un petit compagnon qui réagit aux interactions.

# Technique

## Choix de l'API

Le projet s'appuie sur l'API publique **TCGdex (https://tcgdex.dev)**, une API REST open source, gratuite et sans authentification, référençant plus de 130 000 cartes, disponibles en plusieurs langues (dont le français). Aucune clé n'est nécessaire.

Elle expose notamment **GET https://api.tcgdex.net/v2/en/cards** (liste des cartes, filtrable par champ, ex. **?name=pikachu**, triable via **sort:field/sort:order**, paginable via **pagination:page/pagination:itemsPerPage**) et **GET https://api.tcgdex.net/v2/en/cards/:id** (une carte précise).

Chaque carte comprend son **nom**, sa **catégorie**, sa **rareté**, ses **variantes** (normale, holo, reverse, 1ère édition), les informations de **son set (extension, date de sortie)**, son **illustration**, ainsi que le **Pokémon dont elle évolue** le cas échéant (détail complet en section 9).

## Choix techniques

La stack retenue associe **React** (composants fonctionnels et hooks) pour la structure de l'interface, **Three.js** pour le rendu 3D de la scène étoilée, et un **appel HTTP natif (fetch)** encapsulé dans une couche de service dédiée.

L'architecture suit une séparation stricte entre récupération de la donnée et affichage : **services/tcgdexApi.js** est le seul module qui parle à l'API (schéma détaillé en section 9), **un hook useCards()** centralise l'état (cartes, chargement, erreur, carte sélectionnée), et les composants — ***SkyScene***, ***SearchBar***, ***CardReveal***, ***PokeCompanion*** — ne font que consommer cet état, sans jamais requêter l'API eux-mêmes.

# Moodboard

Moodboard



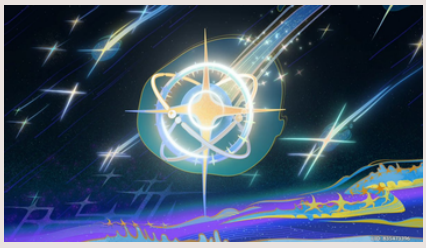










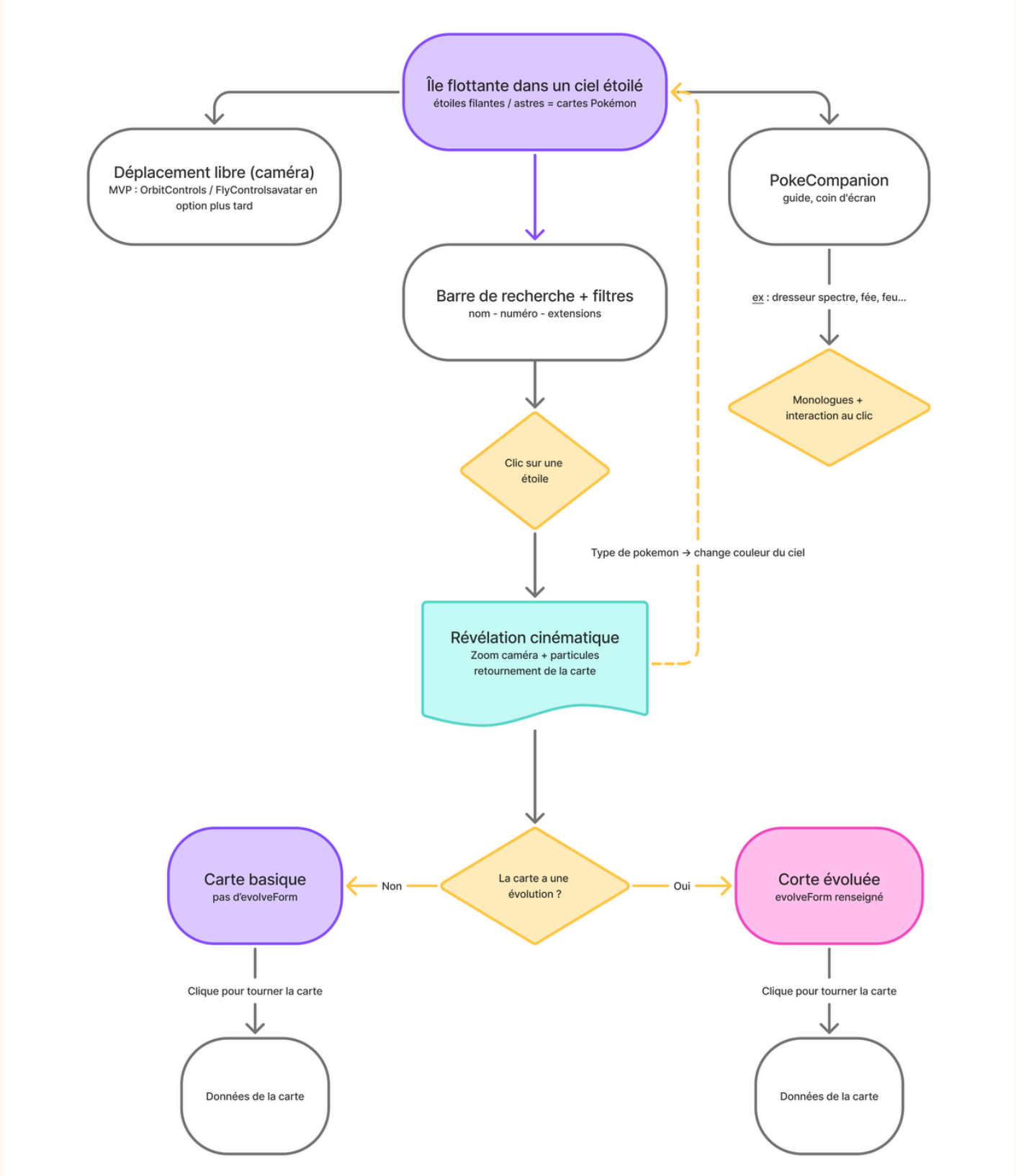







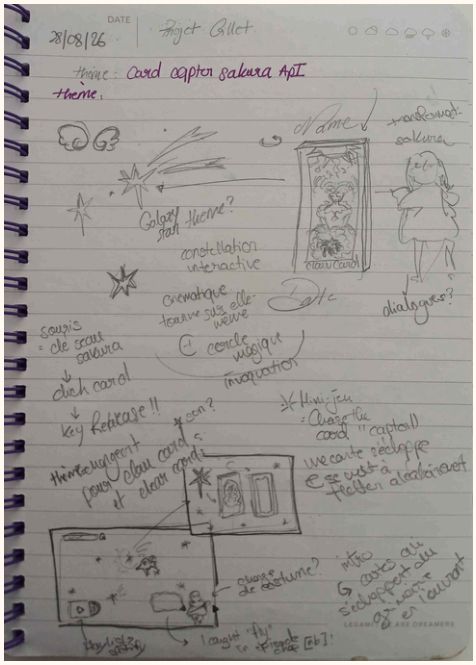


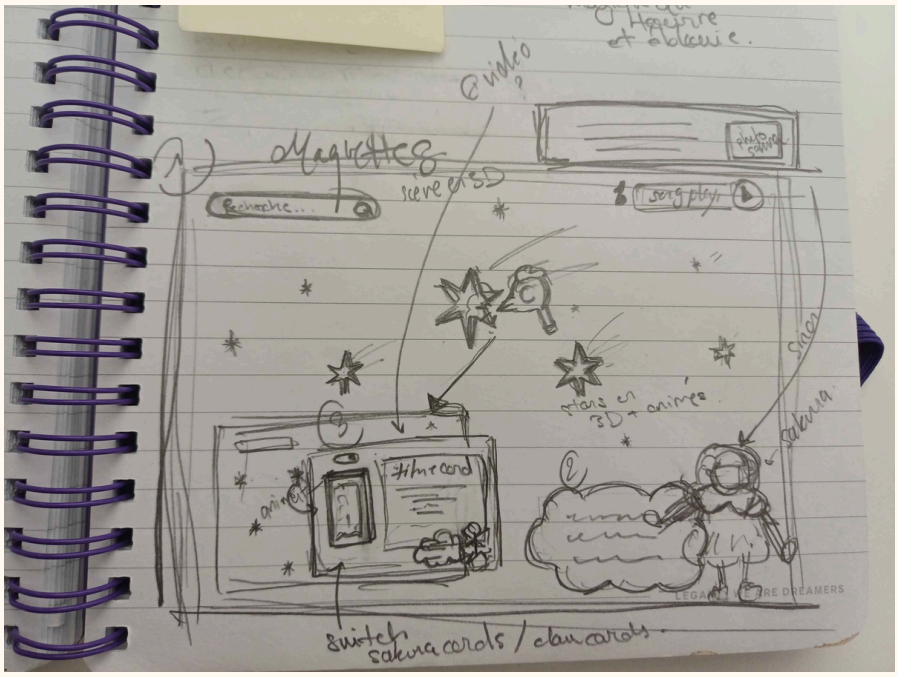

# Arborescence

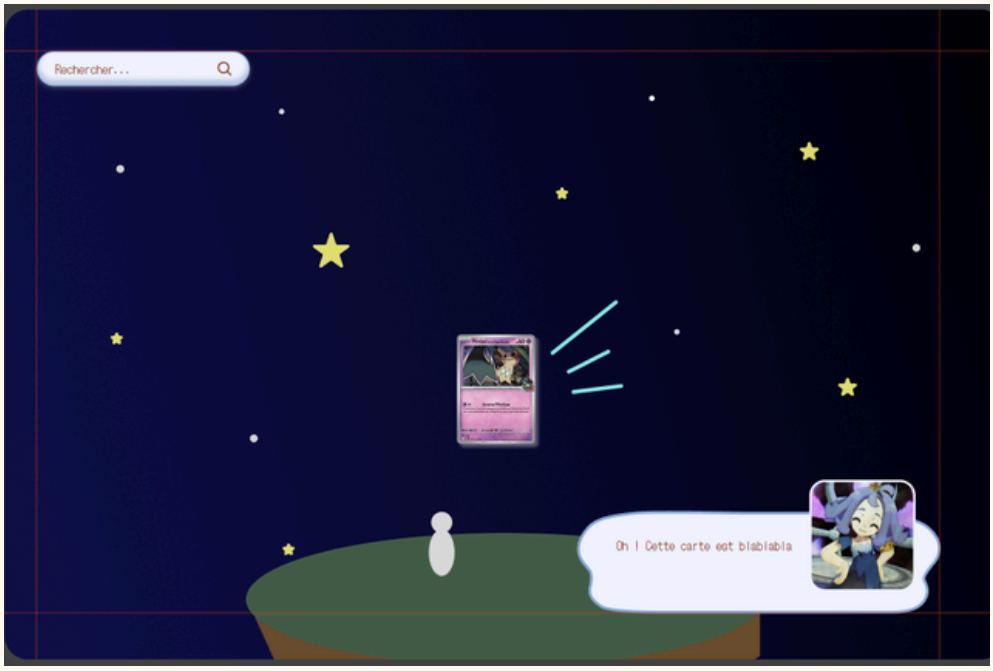



*Lien Arborescence Figma*

# Maquettes







*Lien Maquettes Figma*

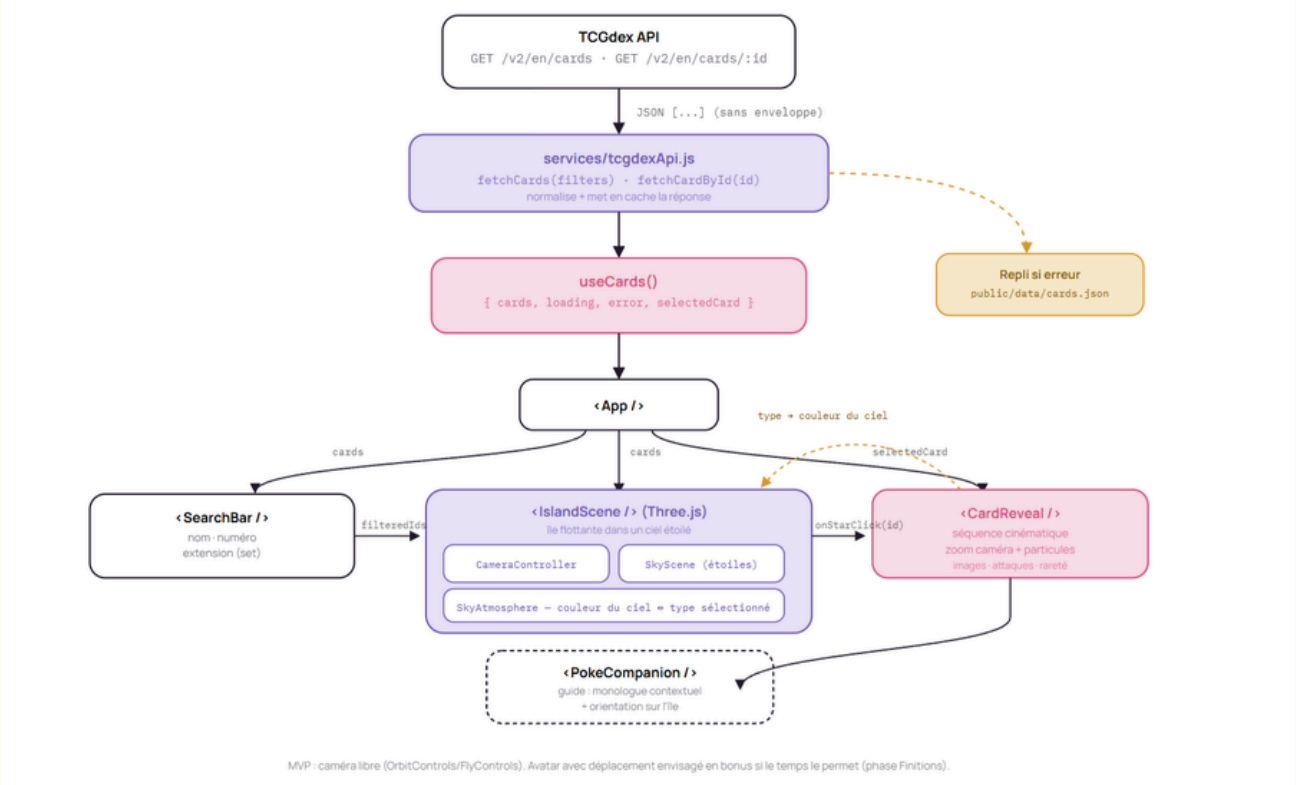
# Données

## Schéma d'intégration des données de l'API

La couche **services/tcgdexApi.js** est le seul point de contact avec l'API : elle appelle **GET /v2/en/cards** (filtrable, triable et paginable via paramètres de requête) et **GET /v2/en/cards/:id**, et bascule sur une copie JSON locale versionnée dans le repo si l'API est indisponible ou renvoie une erreur. **Le hook useCards()** est l'unique source de vérité côté React : aucun composant enfant ne fetch lui-même.

**SearchBar** filtre les cartes visibles, **IslandScene** matérialise l'île flottante et sa voûte étoilée — via ses trois responsabilités internes **CameraController** (navigation libre), **SkyScene** (une étoile = une carte) et **SkyAtmosphere** (couleur du ciel selon le type Pokémon sélectionné).

Un clic sur une étoile déclenche **CardReveal**, la séquence cinématique de révélation, qui informe à son tour **PokeCompanion** et renvoie le type de la carte à **SkyAtmosphere** pour changer l'ambiance du ciel.




*Flux de données de l'API vers les composants React de Pocket Stars.*



*Version interactive du schéma : artifact Pocket stars— Intégration API*

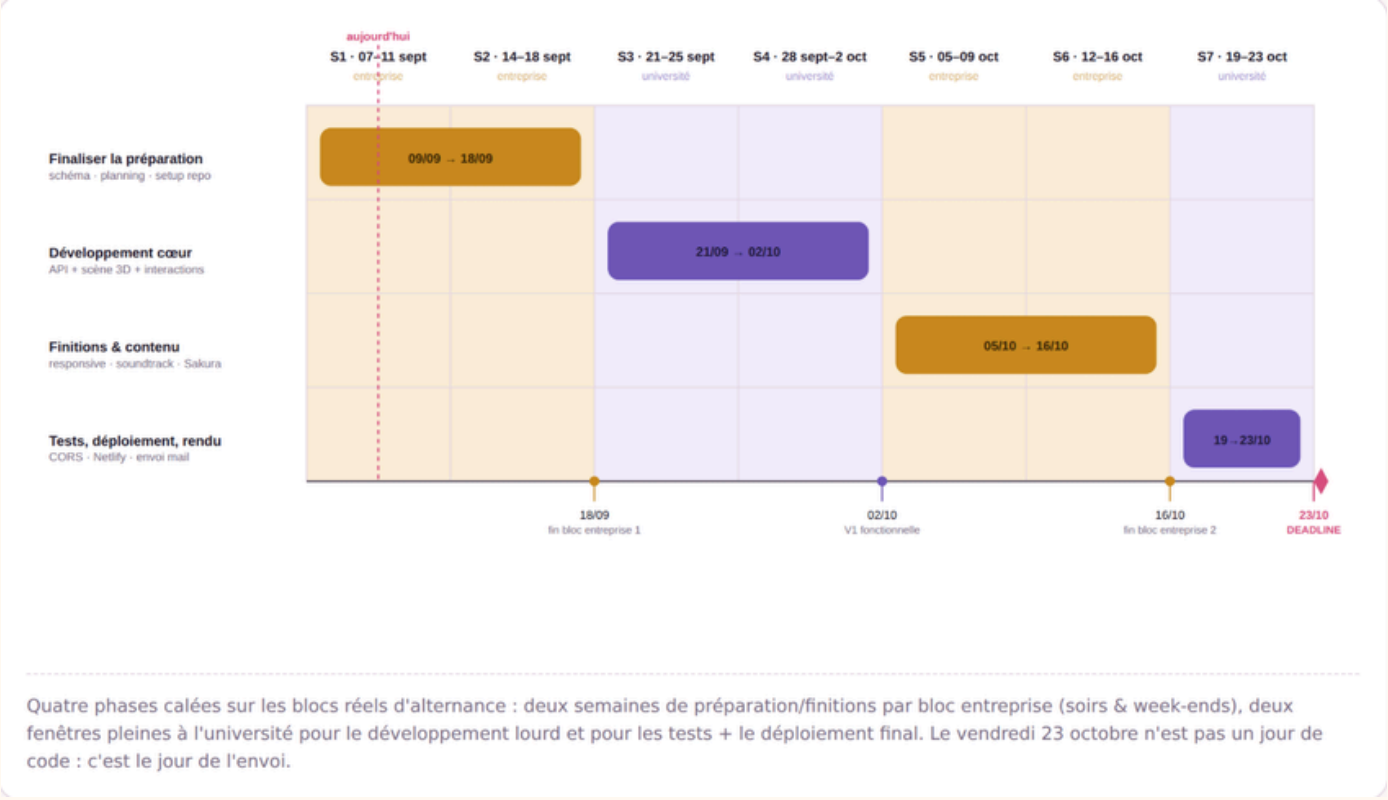
# Données

## Contrat de l'API

Deux endpoints publics, sans clé ni authentification : **GET /v2/cards** (liste paginée, paramètres page et pageSize) et **GET /v2/cards/:id** (une carte précise). La réponse est un objet JSON **{ data: Card[] }**.

| Champ | Type | Usage dans l'app |
|---|---|---|
| id | string | Clé de sélection (étoile ↔ carte), clé React des listes. |
| localId | string/number | Numéro affiché sur la carte physique, tri secondaire. |
| name | string | Titre affiché dans CardReveal et indexé par SearchBar. |
| category / rarity | array / string | Bloc d'infos complémentaire de la fiche de carte. |
| set.name / set.releaseDate | string | Repères « extension / date de sortie » affichés en sous-texte. |
| image (+ suffixe .png/.webp) | url | Textures appliquées à l'étoile 3D + illustration dans CardReveal. |
| evolveFrom | string / array | Détermine si un bouton « évolution » est affiché |

# Planning




Quatre phases calées sur les blocs réels d'alternance : deux semaines de préparation/finitions par bloc entreprise (soirs & week-ends), deux fenêtres pleines à l'université pour le développement lourd et pour les tests + le déploiement final. Le vendredi 23 octobre n'est pas un jour de code : c'est le jour de l'envoi.

*Planning réaliste de Clowleaf, aligné sur le rythme d'alternance.*

*Version interactive du planning : artifact — Planning alternance*

## Risques et marges de sécurité

- **Tester la connexion à l'API** — et le comportement CORS — **dès la semaine 1,** pas à la dernière semaine : un blocage découvert tôt laisse le temps de préparer le repli JSON local.
- **Ne plus ajouter de fonctionnalité après le 16 octobre** : la dernière semaine, la plus courte et la plus contrainte, est réservée aux tests et au déploiement.
- **Réserver les semaines en entreprise aux tâches qui se découpent bien en sessions courtes** (contenu, style, corrections), et **les semaines à l'université aux tâches qui demandent de la concentration continue** (architecture, intégration API).
- **Anticiper l'obsolescence commerciale d'une API tierce** : **Pokémon TCG API** a annoncé sa propre fin de vie au profit d'un service payant (Scrydex, dès 29 $/mois). Bascule vers **TCGdex** (open source, gratuite, sans risque de paywall identifié) réalisée le 16/09, avant tout développement dépendant de l'ancienne API.

# Planning

| Phase | Dates | Contexte | Contenu |
|---|---|---|---|
| **Finaliser la préparation** | 09/09 → 18/09 | Entreprise | Schéma d'intégration, planning, setup repo GitHub + projet React/Three.js, premier test d'appel API (CORS). |
| **Développement cœur** | 21/09 → 02/10 | Université | Service API + hook useCards(), scène 3D (ciel étoilé), interactions (clic → révélation), barre de recherche. |
| **Finitions & contenu** | 05/10 → 16/10 | Entreprise | compagnon, responsive, corrections des bugs remontés, rédaction du README. |
| **Tests, déploiement, rendu** | 19/10 → 23/10 | Université | Tests croisés, déploiement Netlify, recette finale, envoi du rendu le vendredi 23/10. |

Dossier de préparation

# Merci Beaucoup

Camélia Amin Handoyo

2026